

COSC 4P80 Assignment 2 Report

Nestor Voronych

November 2025

1 Introduction

The neural network implemented for this assignment has a three-layer structure: the input layer, the hidden layer, and the output layer. In practice, the implementation consists of four classes that handle the feed-forward network itself, plus one class for backpropagation and a main class that runs the network's training and uses it once it is trained. The four classes that make up the feed-forward portion of the network, in the order that data flows through them, are `InputToHiddenLayer`, `HiddenLayer`, `InputToOutputLayer`, and `OutputLayer`.

Before data can be used for training and testing, it must first be processed in the `Main` class. To process the data from a file, the `Main` class performs the following steps: it reads the file line by line into a two-dimensional matrix, where each row begins with the motor's y-label (verdict), followed by that motor's data values. From there, the matrix is separated into two parts — a matrix containing only the motor data, where each row represents an input vector, and a list of verdicts (y-labels of 1 or 0), where each index corresponds to the matching data row in the input matrix.

The next step standardizes the list of verdicts to fit the two-output-neuron architecture. This involves converting the list into a two-column matrix where each row contains either (1, 0) or (0, 1), a format ready for calculating the error of each of the two output neurons since each column corresponds to one neuron. The final step standardizes the input data matrix by squashing all input values into the 0–1 range using the sigmoid function, producing normalized vectors of data as the rows of the matrix.

The forward pass begins at the input layer, represented by the `InputToHiddenLayer` class. It takes the data representing a motor in the form of a vector. The inputs are multiplied by the weights that correspond to each hidden neuron, producing a vector of weighted sums. This vector is generated by multiplying the input vector by each column of weights associated with the hidden neurons. The size of the resulting weighted vector equals the number of hidden neurons in the network. This weighted vector is then passed to the `HiddenLayer` class, which applies the activation function to it. The resulting activated vector is sent to the `InputToOutputLayer` class. The role of this class is similar to that of the `InputToHiddenLayer` — it produces the weighted input for the `OutputLayer` class. The `OutputLayer` class applies the activation function to the weighted input it receives from the `InputToOutputLayer`, producing a final output vector of

length two, representing the two output neurons. If the output predicts class 1, it means the network classifies the motor as good, and if it predicts class 2, the motor is classified as bad.

2 Experiment Setup

Changes to the initial feed-forward network will be introduced across Parts A to F of the task. Each modification is evaluated to see how it improves the network's ability to learn efficiently—minimizing loss during training while maintaining good generalization rather than memorizing the data.

In Part A, the experiment focuses on finding an optimal learning rate(α). The assumption is that if α is set too high, weight updates will overshoot the minimum of the loss function, causing unstable or divergent learning. Conversely, if α is too small, the network's weight adjustments will be so minor that training will either progress extremely slowly or stall entirely, leading to poor learning. It is also assumed that there exists a “sweet spot” for α that balances convergence speed and stability. However, this assumption may not hold for all data files since the ideal α can vary depending on different parameters which include the number of data points in the input, normalization, sample diversity, network size as well as choice of activation function.

In Part B, the experiment focuses on evaluating the benefits of using k-Fold standardized data to determine whether one fold performs better than another. Each fold contains the same proportion of good and bad motors, ensuring balanced data distribution. Three tests are conducted, where two folds are combined to form the training set and the remaining fold serves as point of comparison. The expectation is that the combined training data, having a broader representation of the overall dataset, will reduce the learning error and allow the network to converge more effectively compared to training on a smaller, single-fold testing dataset

Part C introduces momentum into the network during backpropagation. This addition is intended to help the network learn more quickly and converge faster by accelerating updates in directions that consistently reduce error. However, excessive momentum can have the opposite effect—causing the network to overshoot the minimum of the loss function and destabilize learning instead of improving it.

Part D examines how data granularity affects the network's learning performance. Specifically, it investigates whether increasing the number of bins—thereby providing more detailed input data—improves separability and leads to better training results. However, a higher number of bins may also increase the risk of overfitting, allowing the network to perform well on the training data but generalize poorly when evaluated on the test set.

Part E will test if adding more layers helps the network learn better when we go from 3 to 5 layers. By expanding it from three to five layers, we look to see if the deeper network can find more patterns and avoid getting stuck while learning. It will show if the added hidden layers helps the network generalize better or simply make it a slower and less efficient learning

process.

3 Part A

For the architecture of the network, the alpha landed on is static at a rate of 0.1. This seemed to help the network converge in the least amount of epochs, although increasing both alpha to 0.2 and the number of hidden neurons to 1/2 of the length of the input vector seemed to provide a comparable result. Using an activation function (NOT STANDARDIZING) on the input vectors helped the network train and converge; Using Sigmoid served as a form of normalization (later Network started to use proper normalization) This, along with increasing or decreasing alpha and the number of hidden nodes, increasing the number of epochs, helped the network converge on vectors with 64, 100, and 150 bins in the least number of epochs, while data sets with 16, 25, and 32 bins could not train to an acceptably low error rate with the initial architecture

The number of input and output neurons in the network is predetermined. The number of input neurons corresponds directly to the number of bins per motor, meaning the data file dictates the size of the input layer. The output layer always contains two neurons, representing the two possible classes: 0 for good motors and 1 for bad motors. This leaves only the hidden layer open for adjustment. Most tested values for the number of hidden neurons performed similarly, except when the number of hidden nodes approached that of the input layer. In those cases, the hidden layer began to memorize the training data, reducing its ability to generalize effectively.

4 Part B

Separating the data into training and testing sets is handled by a dedicated class, `SeperateIn3EvenKFolds`. As the name suggests, it splits the input matrix and their y-labels into three folds, each preserving the same good/bad motor proportion; each fold is then shuffled so bad motors aren't clustered at the end of an epoch. In the end, the class outputs KFold 1, KFold 2, and KFold 3.

To produce three graphs of global error (training vs training on test) over epochs, we train on the two-fold combination and test on the remaining fold:

- Train Set 1 = KFold1 + KFold2, vs fold 3 (Figure 1)
- Train Set 2 = KFold1 + KFold3, vs fold 2 (Figure 2)
- Train Set 3 = KFold2 + KFold3, vs fold 1 (Figure 3)

What was unexpected in the results is that, in two out of three K-Fold combinations, the loss using the test set decreased faster and further than in the combined training set. This contradicts the initial assumption that a larger amount of training data would lead to quicker learning. These results suggest that the diversity of the data contributes more to effective classification

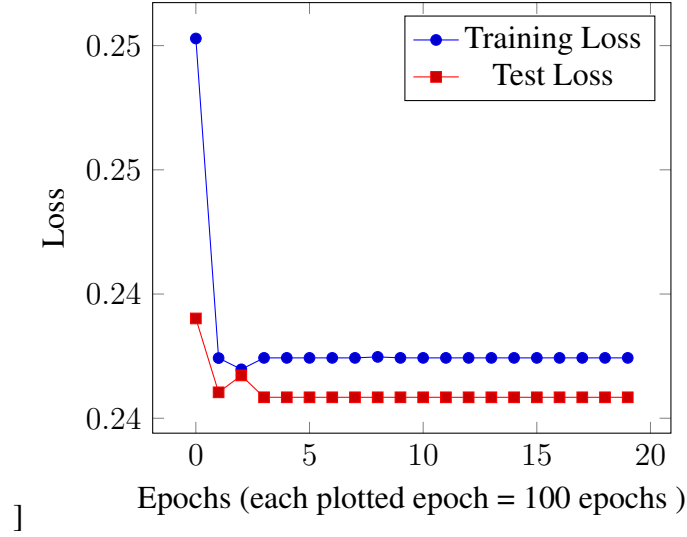


Figure 1: GlobalError of A+B K-Fold vs C-Fold

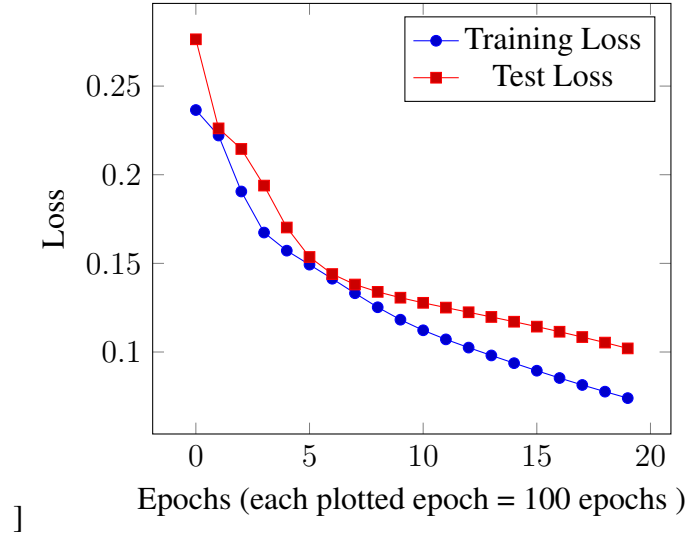


Figure 2: GlobalError of A+B K-Fold vs C-Fold
WITH NORMALIZATION

between “good” and “bad” motors than the sheer quantity of data, as more data can sometimes cause the network to memorize patterns rather than generalize. However, this pattern did not hold across all datasets. Once normalization was added before training, and the network was no longer getting stuck almost immediately at a local minimum, the graphs (Graphs 2, 4, and 6) showed that the combined data provided better representation and trained to a lower error. This is likely due to how few (x, y) motor samples the network has to learn from—at such a small sample size, the quality and variability of samples matter more than quantity alone.

The most effective configuration was achieved using one-fifth of the input vector length as the number of hidden neurons. During experimentation, other configurations were tested, but most produced similar results, with differences within the margin of error. However, when the number of hidden neurons approached half of the input vector length, the network required

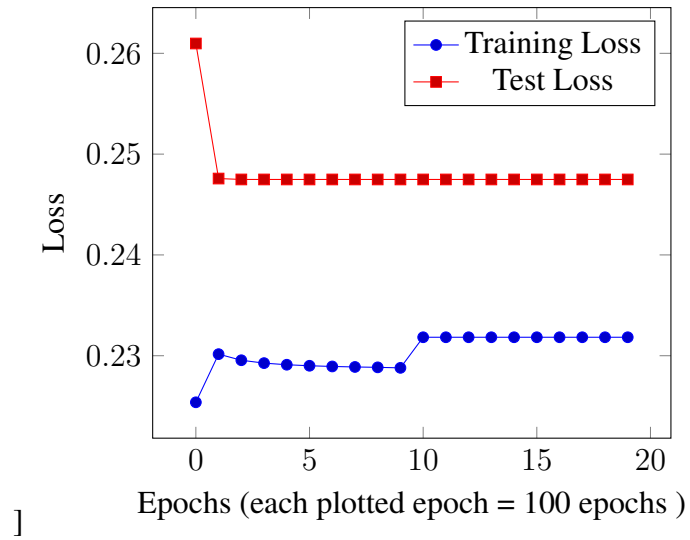


Figure 3: GlobalError of A+C K-Fold vs B-Fold

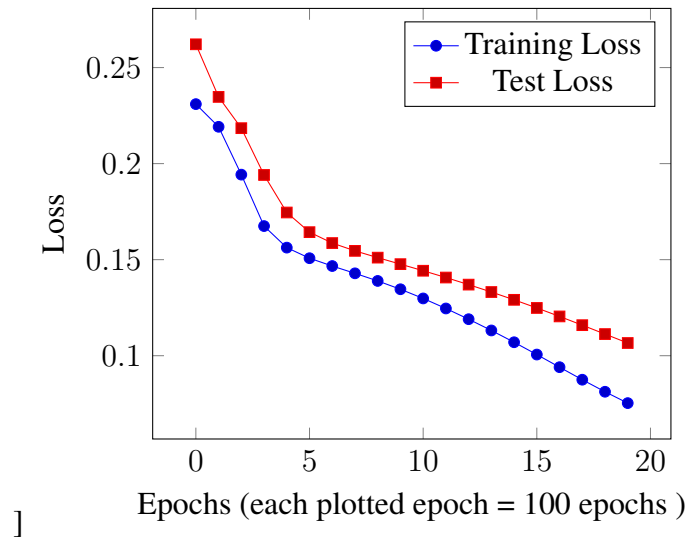


Figure 4: GlobalError of A+C K-Fold vs B-Fold
WITH NORMALIZATION

noticeably more epochs to reach a training loss below 0.05.

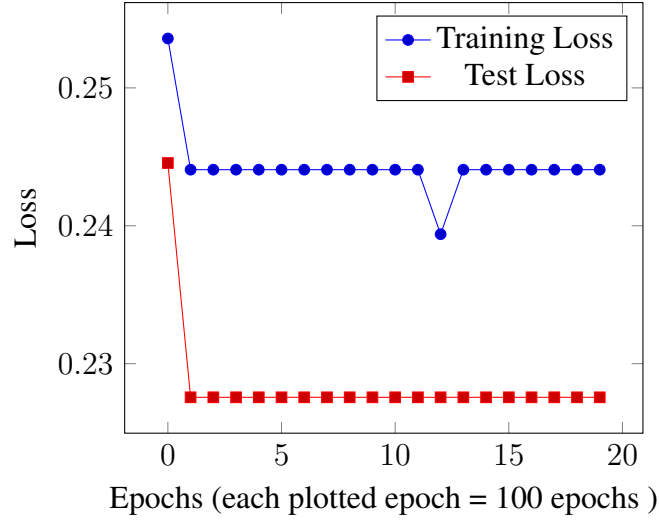


Figure 5: GlobalError of B+C K-Fold vs A-Fold

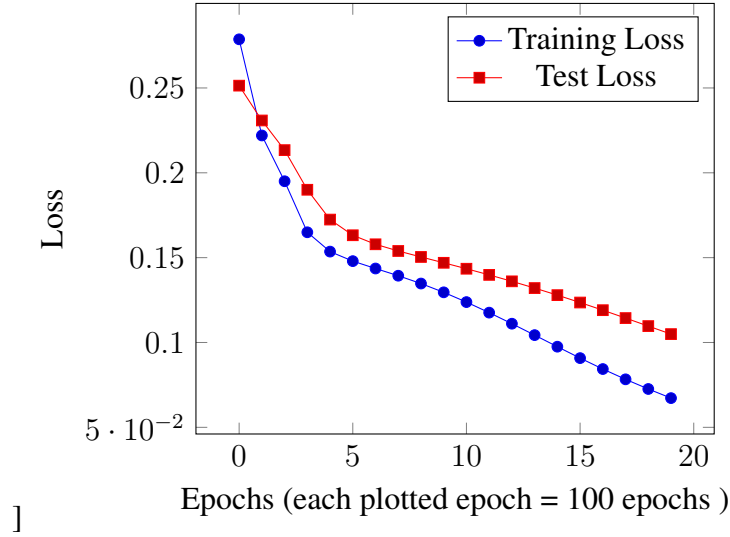


Figure 6: GlobalError of B+C K-Fold vs A-Fold
WITH NORMALIZATION

5 Part C

In Part Three, the network introduces momentum during training. This helps it reach the optimal solution faster by making larger adjustments when the global error decreases and smaller ones when the error increases. The momentum for each weight is stored in 2D matrices that are the same size as the weight matrices. It is calculated based on the change in loss between epochs, with each weight's update stored by multiplying it by *(the previously calculated momentum and adding + the change for that weight (deltaW).) From the collected data, graphs were generated to provide deeper insight into the network's learning behavior when momentum was applied. Initially, momentum appeared to be the key missing factor—the network's loss decreased rapidly toward zero, with each epoch contributing to steady improvement. In several instances, the larger update steps introduced by momentum enabled the network to

minimize loss more quickly than training without it. A closer examination of the graphs and numerical results revealed that for the A+B K-Folds, the network without momentum reached a loss of 0.1 after approximately 1500 epochs, whereas the version with momentum achieved the same level of error in only 1000 epochs. Similarly, achieving a loss of 0.07 required around 2000 epochs without momentum but fewer than 1500 with it. However, this advantage was less pronounced in the B+C folds, where both approaches reached a loss of approximately 0.7.

More notably, the normalization of input data had a far greater impact on training performance than the inclusion of momentum. Without normalization, the network tended to stagnate early, showing little to no improvement after a brief period of training. Figures 8 and 10 illustrate the network's unstable learning behavior when momentum was applied without normalization, whereas Figures 7 and 9 show the corresponding results for the same K-Fold combinations with normalization applied. The comparisons here highlight the critical role of normalization in ensuring learning stability

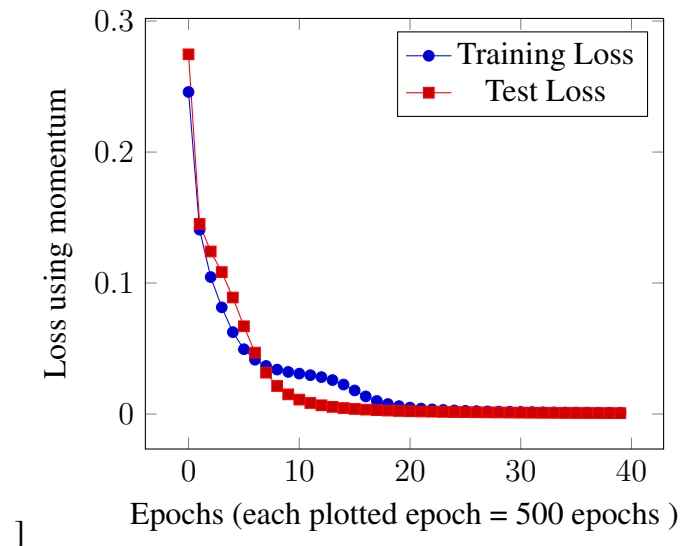


Figure 7: GlobalError of A+B K-Fold vs C-Fold when momentum is added

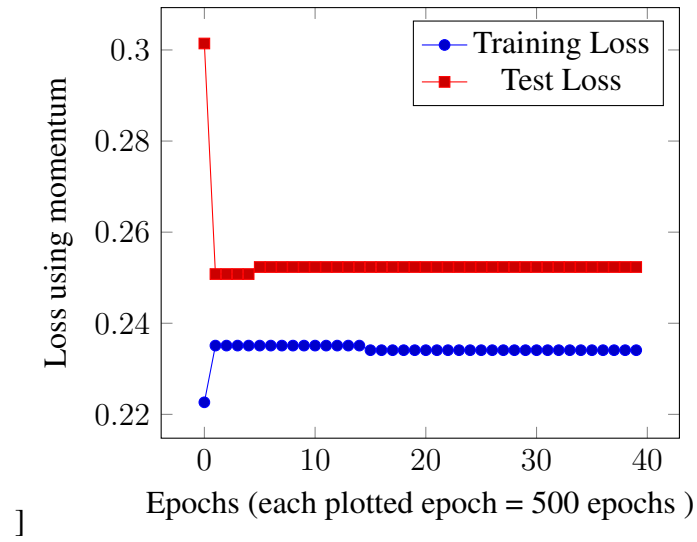


Figure 8: GlobalError of A+B K-Fold vs C-Fold when momentum is added without normalization

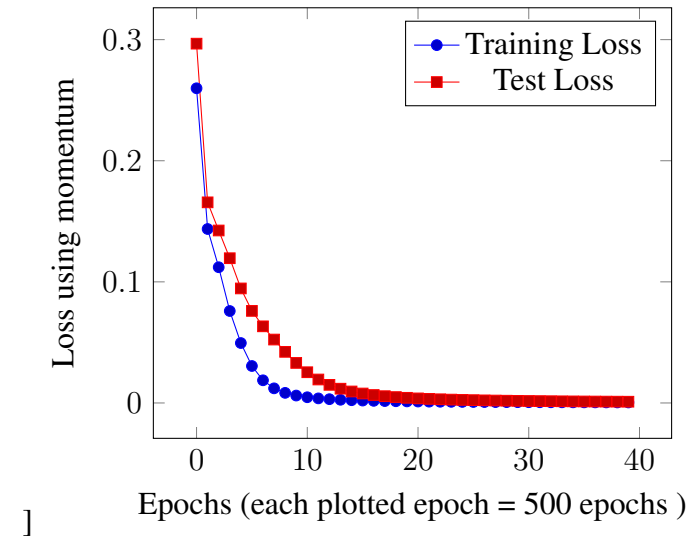


Figure 9: GlobalError of A+C K-Fold vs B-Fold when momentum is added

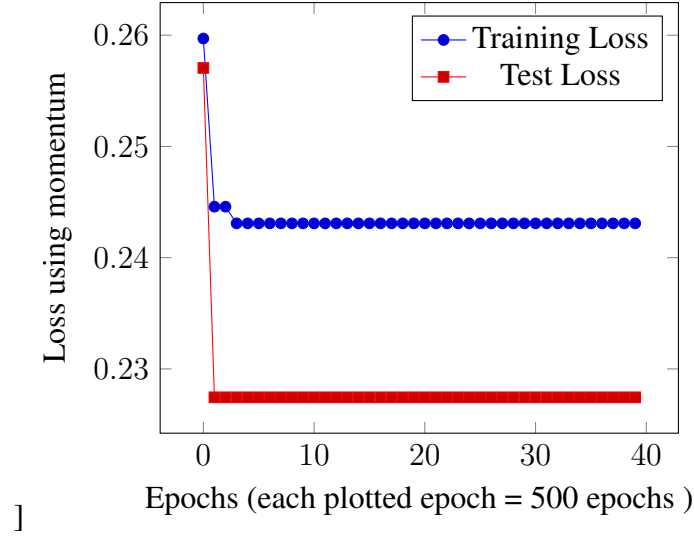


Figure 10: GlobalError of A+C K-Fold vs B-Fold when momentum is added
WITHOUT NORMALIZATION

6 Part D

Table 1: Final Training Losses after training sets with 36,150 and 100 bins followed by the average loss across 5 runs

Dataset (Bins)	Training Sets	Test Set	Train 1	Train 2	Train 3	Train 4	Train 5	Train Avg
32	A+B	C	0.0010	0.0012	0.0011	0.0011	0.0010	0.0010
32	A+C	B	0.0296	0.0298	0.0296	0.0298	0.0296	0.0297
150	A+B	C	3.18×10^{-4}	3.17×10^{-4}	3.15×10^{-4}	3.18×10^{-4}	3.17×10^{-4}	3.17×10^{-4}
150	A+C	B	1.69×10^{-4}	1.70×10^{-4}	1.69×10^{-4}	1.68×10^{-4}	1.68×10^{-4}	1.69×10^{-4}
1000	A+B	C	9.11×10^{-5}	9.17×10^{-5}	9.13×10^{-5}	9.12×10^{-5}	9.14×10^{-5}	9.14×10^{-5}
1000	A+C	B	6.728×10^{-5}	6.763×10^{-5}	6.731×10^{-5}	6.719×10^{-5}	6.783×10^{-5}	6.745×10^{-5}

Table 2: Test Losses for C and B for Datasets with varying number of bins after training on A+B or A+C

Dataset (Bins)	Training Sets	Test Set	Test 1	Test 2	Test 3	Test 4	Test 5	Test Avg
32	A+B	C	0.1342	0.1320	0.1084	0.1304	0.1320	0.1340
32	A+C	B	0.1487	0.1540	0.1783	0.1521	0.1487	0.1230
150	A+B	C	0.0794	0.0791	0.0785	0.0785	0.0794	0.0792
150	A+C	B	0.1312	0.1314	0.1314	0.1307	0.1308	0.1311
1000	A+B	C	0.0032	0.0017	0.0018	0.0027	0.0032	0.0025
1000	A+C	B	0.1282	0.1462	0.1218	0.1451	0.1384	0.1360

7 Part E

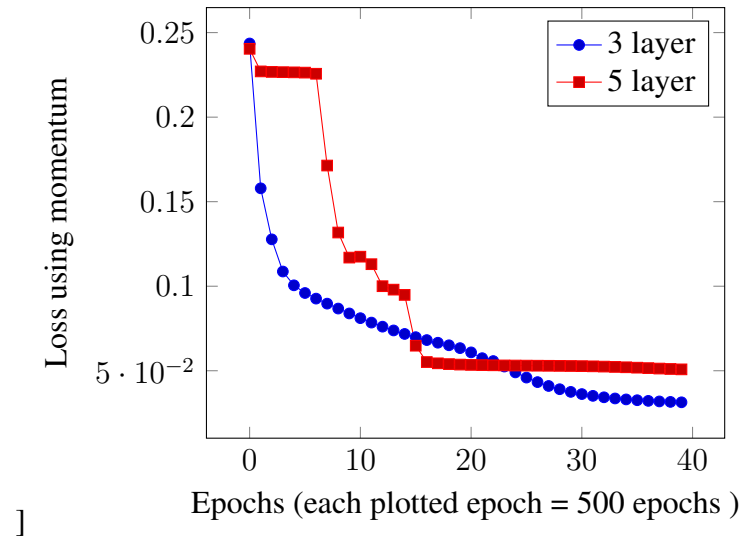


Figure 11: Training on 16bin data
3 layer network vs 5 layer network

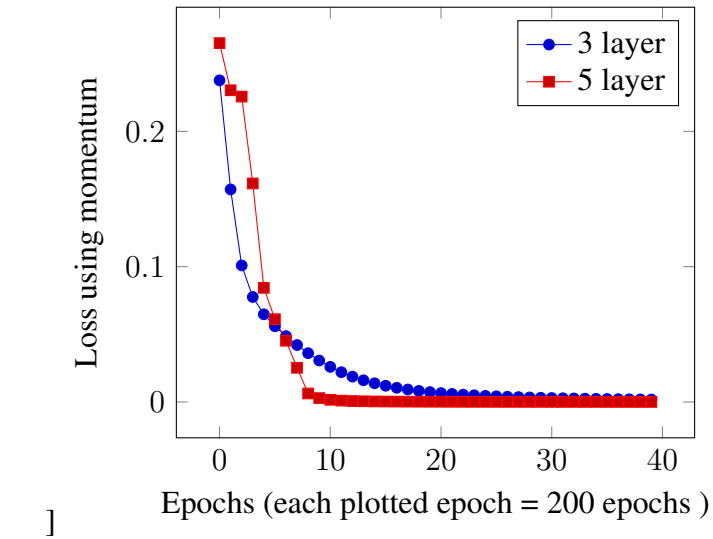


Figure 12: Training on 150bin data
3 layer network vs 5 layer network

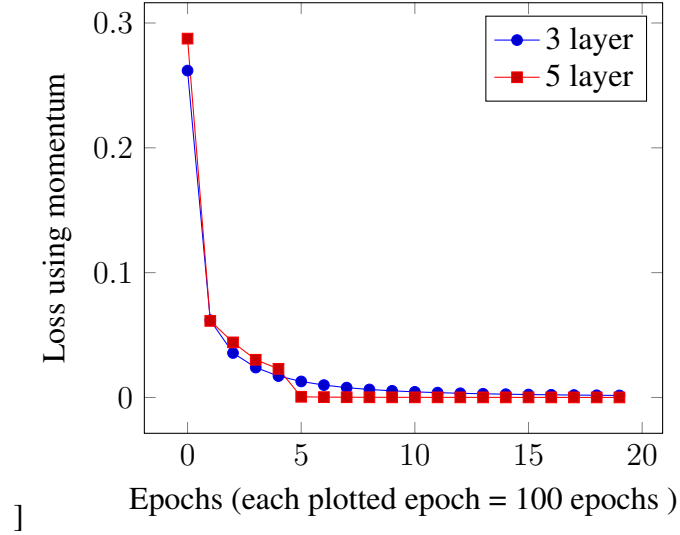


Figure 13: Training on 1000 bin data
3 layer network vs 5 layer network

In Part E, we extended the network to have 5 layers. This should, in theory, allow the model to capture more complex patterns, with each hidden layer becoming responsible for learning its own set of features. It could also help the network avoid getting stuck in local minimums. However, looking at the graphs, there was no noticeable improvement from adding the extra two hidden layers. If anything, they made the performance worse. The test results showed the same trend as the training results: for the 1000-bin dataset, the 5-layer network achieved a final loss of 0.13350 compared to 0.045857 for the 3-layer network, and for the 16-bin dataset, 0.17556 versus 0.1622. This drop in performance is likely due to having the same number of neurons in each hidden layer or the nature of the data itself, which may not benefit from a deeper architecture.

8 Conclusion

In summary, the series of experiments demonstrated how each modification to the network architecture and training process influenced learning stability, convergence speed, and generalization. Normalization proved to be the single most impactful factor in enabling consistent learning, while momentum provided faster convergence without significantly improving final accuracy. Increasing data granularity improved separability up to a point, which overfitting became more likely (going from 150 to 1000). Extending the network to five layers did not yield better performance, suggesting that deeper architectures without a proportional decrease in neurons per layer can hinder training efficiency for smaller datasets. Overall, the three-layer normalized network with dynamic momentum achieved the best balance between speed and accuracy for predicting whether a motor is good or bad.